Homework 1

Lixi Zhang
Student ID: 2025011067

October 21, 2026

Q1

Q1.1 Search with Heuristics

(a) Size of the state space

A state assigns the n labelled cars to n distinct cells of the $n \times n$ grid, so the exact number of states is

$$\binom{n^2}{n} n! = n^2(n^2-1)\cdots(n^2-n+1) = \Theta(n^{2n}),$$

i.e. $O(n^{2n})$.

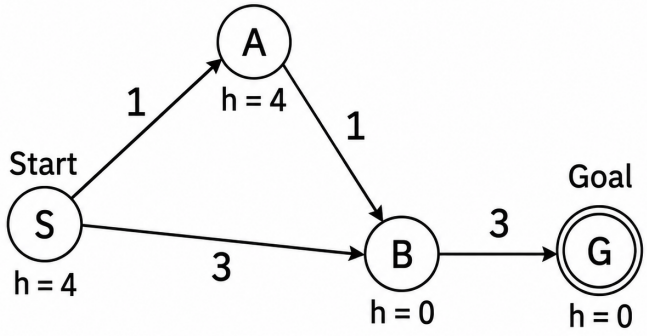
(b) Admissibility of h_i

With a single car the grid is empty, so the shortest path to the goal is exactly the Manhattan distance and h_i is admissible. With the other $n-1$ cars on the grid it is *not* admissible: a stationary car can be hopped over, covering two cells at cost 1, so the true cost can be smaller than h_i.

(c) Admissible candidates

candidate	admissible	reason
(i) $\sum_i h_i$	no	$n=3$, cars $(1,1),(1,2),(1,3)$: $\sum_i h_i = 6$, true cost 5
(ii) $\min_i h_i$	**yes**	each car needs at least $\lceil h_j/2 \rceil$ moves, hence $h^* \geq n\lceil \min_i h_i/2 \rceil \geq \min_i h_i$
(iii) $\max_i h_i$	no	$n=3$, cars $(1,1),(2,3),(1,2)$: $\max_i h_i = 4$, true cost 3
(iv) $(h_1,\ldots,h_n)$	no	componentwise it inherits (b) — a hop covers two cells at cost 1, so h_i may exceed the cost of car i; a vector also gives no total order on the fringe
(v) $n \max_i h_i$	no	$n=3$, cars $(1,1),(2,1),(2,2)$: $n\max_i h_i = 12$, true cost 5

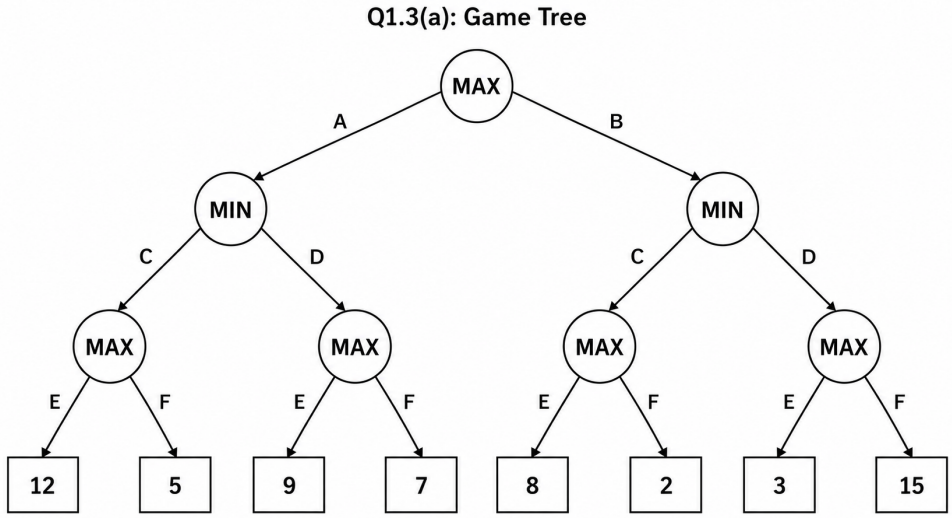
Q1.2 Adjusted Heuristics



Take the graph above with $h(S) = 4$, $h(A) = 4$, $h(B) = 0$, $h(G) = 0$: admissible ($h^* = 5, 4, 3, 0$) but inconsistent on $S \to B$. Expanding S raises $h(B) = \max(0, 4 - 3) = 1$, so B is filled in first by the long path $S \to B$ with $g = 3$ and is expanded before A, i.e. with $g = 3$ instead of $g^*(B) = 2$. When A is expanded afterwards, the shorter path $A \to B$ with $g = 2$ can no longer be extended to B, since B is already closed.

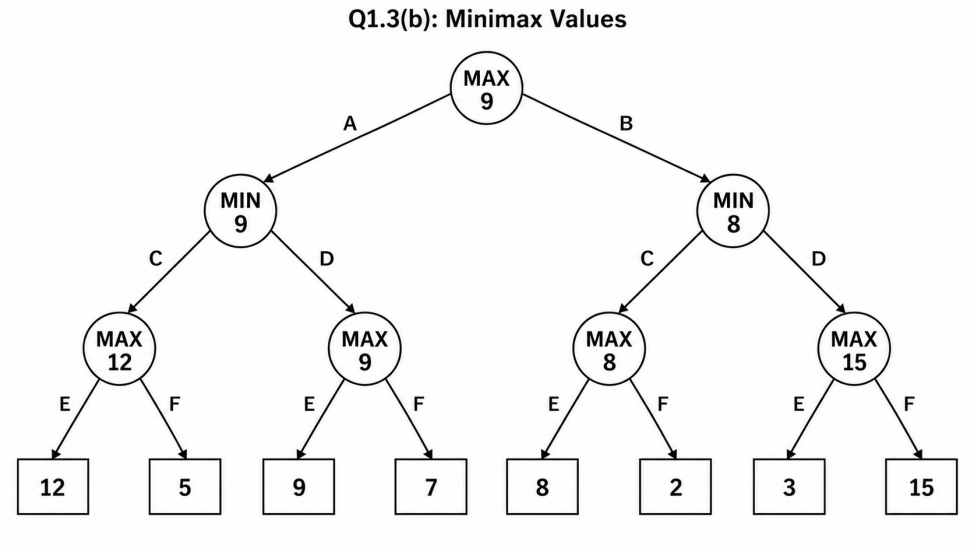
Q1.3 Adversarial Search

(a) Game tree



(b) Minimax value of the root

The minimax value of the root is 9.



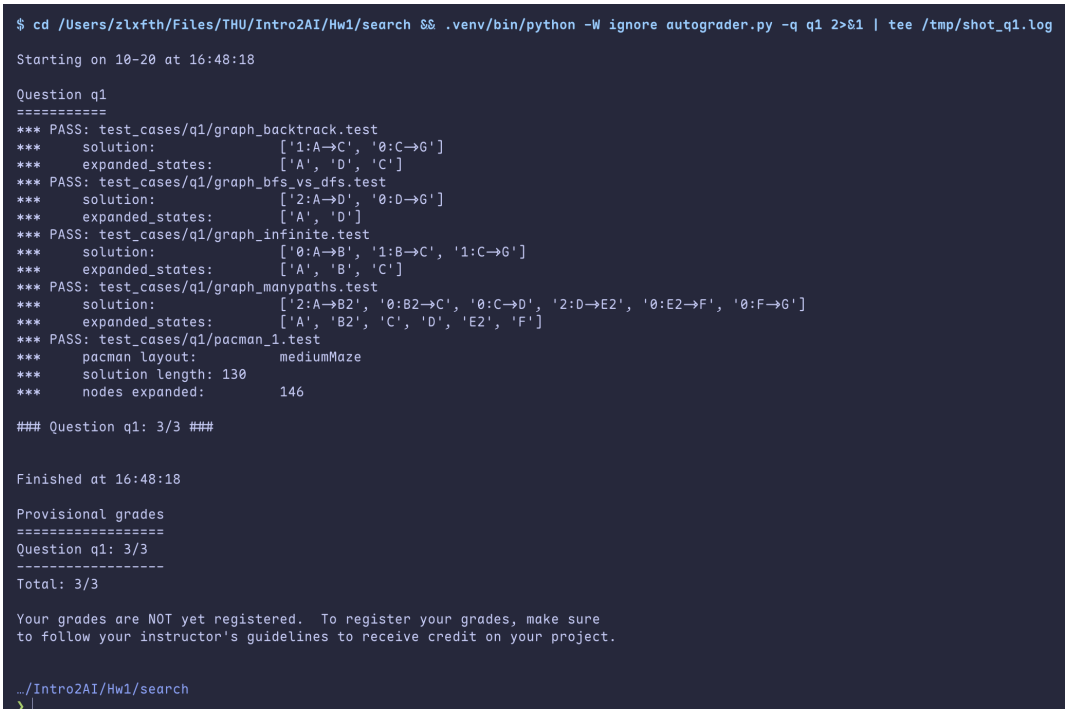
(c) Alpha–beta pruning

Traversing left to right, the last MAX node on the third level is pruned: before it is entered, $\alpha = 9$, while its MIN parent already holds 8.

Q2

Q2.1 Depth-first search

Graph search with a `util.Stack`. On `mediumMaze` it reaches the goal in 130 steps after expanding 146 nodes: the path is legal, but as expected not optimal.



```
$ cd /Users/zlxfth/Files/THU/Intro2AI/Hw1/search && .venv/bin/python -W ignore autograder.py -q q1 2>&1 | tee /tmp/shot_q1.log

Starting on 10-20 at 16:48:18

Question q1
===========
*** PASS: test_cases/q1/graph_backtrack.test
***     solution:               ['1:A→C', '0:C→G']
***     expanded_states:        ['A', 'D', 'C']
*** PASS: test_cases/q1/graph_bfs_vs_dfs.test
***     solution:               ['2:A→D', '0:D→G']
***     expanded_states:        ['A', 'D']
*** PASS: test_cases/q1/graph_infinite.test
***     solution:               ['0:A→B', '1:B→C', '1:C→G']
***     expanded_states:        ['A', 'B', 'C']
*** PASS: test_cases/q1/graph_manypaths.test
***     solution:               ['2:A→B2', '0:B2→C', '0:C→D', '2:D→E2', '0:E2→F', '0:F→G']
***     expanded_states:        ['A', 'B2', 'C', 'D', 'E2', 'F']
*** PASS: test_cases/q1/pacman_1.test
***     pacman layout:          mediumMaze
***     solution length: 130
***     nodes expanded:         146

### Question q1: 3/3 ###


Finished at 16:48:18

Provisional grades
==================
Question q1: 3/3
------------------
Total: 3/3

Your grades are NOT yet registered.  To register your grades, make sure
to follow your instructor's guidelines to receive credit on your project.


…/Intro2AI/Hw1/search
```

Q2.2 Breadth-first search

The same skeleton with a `util.Queue`. BFS does find a least-cost solution: 68 steps on `mediumMaze`, against 130 for DFS, with 269 nodes expanded.

```
$ cd /Users/zlxfth/Files/THU/Intro2AI/Hw1/search && .venv/bin/python -W ignore autograder.py -q q2 2>&1 | tee /tmp/shot_q2.log

Starting on 10-20 at 16:48:25

Question q2
===========
*** PASS: test_cases/q2/graph_backtrack.test
***     solution:               ['1:A->C', '0:C->G']
***     expanded_states:        ['A', 'B', 'C', 'D']
*** PASS: test_cases/q2/graph_bfs_vs_dfs.test
***     solution:               ['1:A->G']
***     expanded_states:        ['A', 'B']
*** PASS: test_cases/q2/graph_infinite.test
***     solution:               ['0:A->B', '1:B->C', '1:C->G']
***     expanded_states:        ['A', 'B', 'C']
*** PASS: test_cases/q2/graph_manypaths.test
***     solution:               ['1:A->C', '0:C->D', '1:D->F', '0:F->G']
***     expanded_states:        ['A', 'B1', 'C', 'B2', 'D', 'E1', 'F', 'E2']
*** PASS: test_cases/q2/pacman_1.test
***     pacman layout:          mediumMaze
***     solution length: 68
***     nodes expanded:         269

### Question q2: 3/3 ###


Finished at 16:48:25

Provisional grades
==================
Question q2: 3/3
------------------
Total: 3/3

Your grades are NOT yet registered.  To register your grades, make sure
to follow your instructor's guidelines to receive credit on your project.


…/Intro2AI/Hw1/search
❯
```

Q2.3 Uniform-cost search

The fringe is a priority queue keyed on g, and the goal test is applied on dequeue, as the `goalAtDequeue` case checks. With unit step costs UCS coincides with BFS; with the exponential costs of `StayEastSearchAgent` (0.5^x, `mediumDottedMaze`) and `StayWestSearchAgent` (2^x, `mediumScaryMaze`) Pacman hugs the east and the west side respectively, so the costs 68, 1 and 6.87×10^{10} all come from the *same* algorithm under three different cost functions.

```
Starting on 10-20 at 16:48:32

Question q3
===========
*** PASS: test_cases/q3/graph_backtrack.test
***     solution:               ['1:A->C', '0:C->G']
***     expanded_states:        ['A', 'B', 'C', 'D']
*** PASS: test_cases/q3/graph_bfs_vs_dfs.test
***     solution:               ['1:A->G']
***     expanded_states:        ['A', 'B']
*** PASS: test_cases/q3/graph_infinite.test
***     solution:               ['0:A->B', '1:B->C', '1:C->G']
***     expanded_states:        ['A', 'B', 'C']
*** PASS: test_cases/q3/graph_manypaths.test
***     solution:               ['1:A->C', '0:C->D', '1:D->F', '0:F->G']
***     expanded_states:        ['A', 'B1', 'C', 'B2', 'D', 'E1', 'F', 'E2']
*** PASS: test_cases/q3/ucs_0_graph.test
***     solution:               ['Right', 'Down', 'Down']
***     expanded_states:        ['A', 'B', 'D', 'C', 'G']
*** PASS: test_cases/q3/ucs_1_problemC.test
***     pacman layout:          mediumMaze
***     solution length: 68
***     nodes expanded:         269
*** PASS: test_cases/q3/ucs_2_problemE.test
***     pacman layout:          mediumMaze
***     solution length: 74
***     nodes expanded:         260
*** PASS: test_cases/q3/ucs_3_problemW.test
***     pacman layout:          mediumMaze
***     solution length: 152
***     nodes expanded:         173
*** PASS: test_cases/q3/ucs_4_testSearch.test
***     pacman layout:          testSearch
***     solution length: 7
***     nodes expanded:         14
*** PASS: test_cases/q3/ucs_5_goalAtDequeue.test
***     solution:               ['1:A->B', '0:B->C', '0:C->G']
***     expanded_states:        ['A', 'B', 'C']

### Question q3: 3/3 ###


Finished at 16:48:32

Provisional grades
==================
Question q3: 3/3
------------------
Total: 3/3

Your grades are NOT yet registered.  To register your grades, make sure
to follow your instructor's guidelines to receive credit on your project.


…/Intro2AI/Hw1/search
❯
```

Q2.4 A* search

The priority is $f = g + h$ with h the Manhattan distance. On `bigMaze` A* expands 549 nodes against 620 for UCS and returns the same optimal path of cost 210. On the almost empty `openMaze` the differences are starker:

algorithm	nodes expanded	path cost
DFS	576	298
BFS	682	54
UCS	682	54
A*	535	54

Without walls Manhattan is almost the exact remaining cost, so A* only expands a narrow cone towards the goal, while BFS/UCS must sweep a whole diamond; DFS simply gets lucky along one corridor and pays for it with a path 5.5× longer.

```
$ cd /Users/zlxfth/Files/THU/Intro2AI/Hw1/search && .venv/bin/python -W ignore autograder.py -q q4 2>&1 | tee /tmp/shot_q4.log

Starting on 10-20 at 16:48:40

Question q4
===========
*** PASS: test_cases/q4/astar_0.test
***     solution:               ['Right', 'Down', 'Down']
***     expanded_states:        ['A', 'B', 'D', 'C', 'G']
*** PASS: test_cases/q4/astar_1_graph_heuristic.test
***     solution:               ['0', '0', '2']
***     expanded_states:        ['S', 'A', 'D', 'C']
*** PASS: test_cases/q4/astar_2_manhattan.test
***     pacman layout:          mediumMaze
***     solution length: 68
***     nodes expanded:         221
*** PASS: test_cases/q4/astar_3_goalAtDequeue.test
***     solution:               ['1:A->B', '0:B->C', '0:C->G']
***     expanded_states:        ['A', 'B', 'C']
*** PASS: test_cases/q4/graph_backtrack.test
***     solution:               ['1:A->C', '0:C->G']
***     expanded_states:        ['A', 'B', 'C', 'D']
*** PASS: test_cases/q4/graph_manypaths.test
***     solution:               ['1:A->C', '0:C->D', '1:D->F', '0:F->G']
***     expanded_states:        ['A', 'B1', 'C', 'B2', 'D', 'E1', 'F', 'E2']

### Question q4: 3/3 ###


Finished at 16:48:40

Provisional grades
==================
Question q4: 3/3
------------------
Total: 3/3

Your grades are NOT yet registered.  To register your grades, make sure
to follow your instructor's guidelines to receive credit on your project.


…/Intro2AI/Hw1/search
❯
```

Q2.5 Corners problem

The state is (position, frozenset of visited corners), the only information the goal test needs; the start state is (startingPosition, ∅). BFS then walks the four corners in the optimal 28 steps on `tinyCorners` and expands 1966 nodes on `mediumCorners` ("just under 2000").

```
$ cd /Users/zlxfth/Files/THU/Intro2AI/Hw1/search && .venv/bin/python -W ignore autograder.py -q q5 2>&1 | tee /tmp/shot_q5.log

Note: due to dependencies, the following tests will be run: q2 q5
Starting on 10-20 at 16:48:47

Question q2
===========
*** PASS: test_cases/q2/graph_backtrack.test
***     solution:               ['1:A→C', '0:C→G']
***     expanded_states:        ['A', 'B', 'C', 'D']
*** PASS: test_cases/q2/graph_bfs_vs_dfs.test
***     solution:               ['1:A→G']
***     expanded_states:        ['A', 'B']
*** PASS: test_cases/q2/graph_infinite.test
***     solution:               ['0:A→B', '1:B→C', '1:C→G']
***     expanded_states:        ['A', 'B', 'C']
*** PASS: test_cases/q2/graph_manypaths.test
***     solution:               ['1:A→C', '0:C→D', '1:D→F', '0:F→G']
***     expanded_states:        ['A', 'B1', 'C', 'B2', 'D', 'E1', 'F', 'E2']
*** PASS: test_cases/q2/pacman_1.test
***     pacman layout:          mediumMaze
***     solution length: 68
***     nodes expanded:         269

### Question q2: 3/3 ###


Question q5
===========
*** PASS: test_cases/q5/corner_tiny_corner.test
***     pacman layout:          tinyCorner
***     solution length:                28

### Question q5: 3/3 ###


Finished at 16:48:47

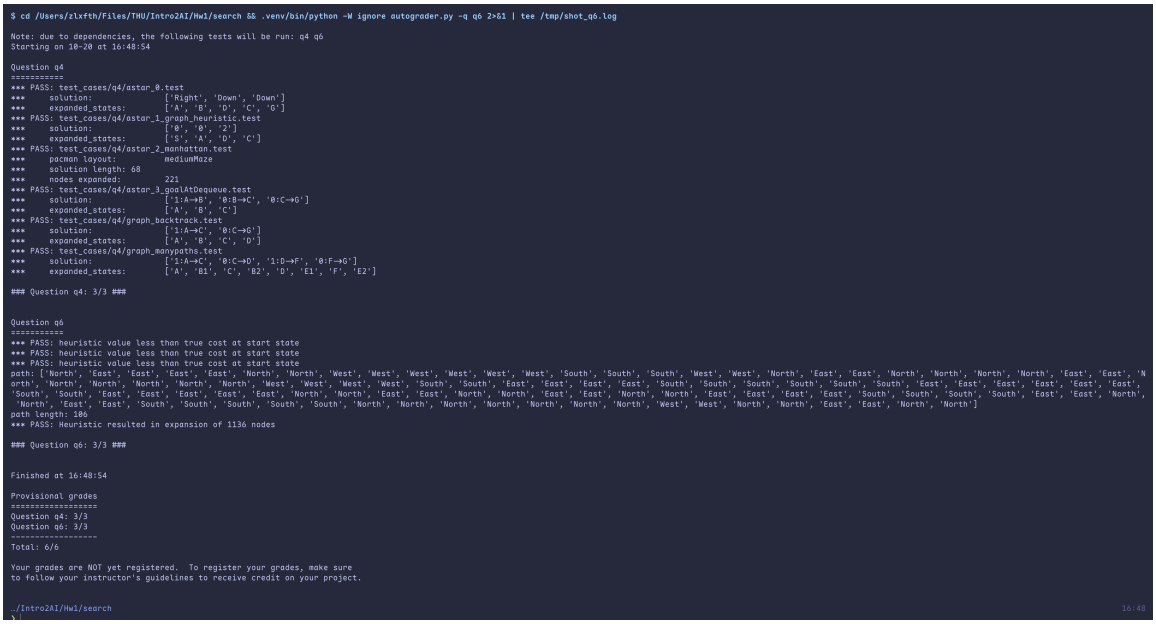
Provisional grades
==================
Question q2: 3/3
Question q5: 3/3
------------------
Total: 6/6

Your grades are NOT yet registered.  To register your grades, make sure
to follow your instructor's guidelines to receive credit on your project.


…/Intro2AI/Hw1/search
```

Q2.6 Corners heuristic

h is the largest Manhattan distance to a corner not yet visited. It is admissible (every remaining corner must be reached, so the farthest one costs at least that much) and consistent (a step changes each distance by at most 1, and stepping onto a corner removes a distance that is then 0). It expands 1136 nodes instead of 1966, a 42% reduction, comfortably below the 1200 threshold for full credit.



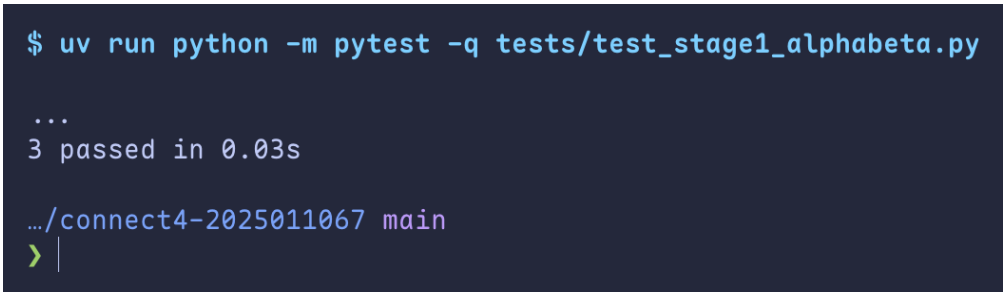
Q3

Q3.1 Alpha–beta search

The score is kept in the fixed `root_player` perspective, terminal states are tested before the depth cutoff, and every successor is generated through `ctx.child`, so the shared budget is charged exactly once per successor. `alpha` is raised when the root player is to move and `beta` is lowered otherwise; the loop breaks as soon as $\alpha \geq \beta$. `BudgetExceeded` is left to propagate to the iterative-deepening wrapper.

depth	minimax successors	Alpha–beta successors	value
2	56	32	6
3	399	147	9
4	2000 (budget exhausted)	523	6

Both searches return the same value at every depth; the saving is what lets depth 4 complete at all inside the 2000-successor budget.



Q3.2 Evaluation heuristic

`evaluate` scores the 69 four-cell windows: an open three for the root player is +100, an open three for the opponent is −200, and a window holding both colours is dead and contributes nothing. Doubling the defensive weight is what removed the baseline's worst decision.

quantity	value
central-column baseline regret	307.0
this heuristic	165.0
reduction	142.0 (46.3%)

The remaining regret sits almost entirely in the fifth benchmark position (121 of 165): two-in-a-row structures and double threats are not scored, so the heuristic cannot tell a move that creates two threats from a move that creates none.

```
$ uv run python -m pytest -q -s tests/test_stage2_evaluation.py


=== Stage 2: evaluation heuristic (6 points) ===
Fixed student depth: 2; deeper reference depth: 4
position_1: action=0 chosen_value=-27.0 best_value=-21.0 regret=6.0
position_2: action=1 chosen_value=-35.0 best_value=-14.0 regret=21.0
position_3: action=1 chosen_value=-160.0 best_value=-152.0 regret=8.0
position_4: action=3 chosen_value=-88.0 best_value=-79.0 regret=9.0
position_5: action=0 chosen_value=12.0 best_value=133.0 regret=121.0
central_column_total_regret=307.0
student_total_regret=165.0
regret_reduction=142.0
suggested_score=6.00/6.00
.
1 passed in 0.03s

…/connect4-2025011067 main
❯
```

Q3.3 Move ordering

Columns are ranked in tiers: immediate win, mandatory block, own open three, opponent open three, own two, then everything else. The line check only reads `state.at` from `state.landing_row(col)` along the four axes, so ordering never calls `ctx.child` and costs no successor budget.

quantity	value
left-to-right baseline accuracy	19.565% (9/46 pairs)
this ordering	73.913% (34/46 pairs)
improvement	+54.348%

Accuracy understates the effect, because better ordering is what makes pruning possible. Swapping only `order_actions` (same search, same heuristic, same depth) gives:

position	depth	left-to-right	this ordering
$(3, 2, 4, 2, 5)$	4	budget exhausted	131
	5	budget exhausted	314
$(3, 2, 3, 2, 4, 1)$	4	732	190
	5	budget exhausted	564
$(3, 1, 4, 2, 4, 2, 5, 3)$	4	807	96
	5	budget exhausted	383

The returned values are identical in every row while the successor count drops by a factor of three to eight, and depths that exhaust the budget under the left-to-right order become reachable.

```
$ uv run python -m pytest -q -s tests/test_stage3_ordering.py


=== Stage 3: move-ordering heuristic (6 points) ===
nonterminal_positions=3
Action labels come from a deeper depth-4 reference search
left_to_right_accuracy=19.565%
student_accuracy=73.913% (34/46 pairs)
accuracy_improvement=+54.348%
suggested_score=6.00/6.00
.
1 passed in 0.01s

…/connect4-2025011067 main
❯
```